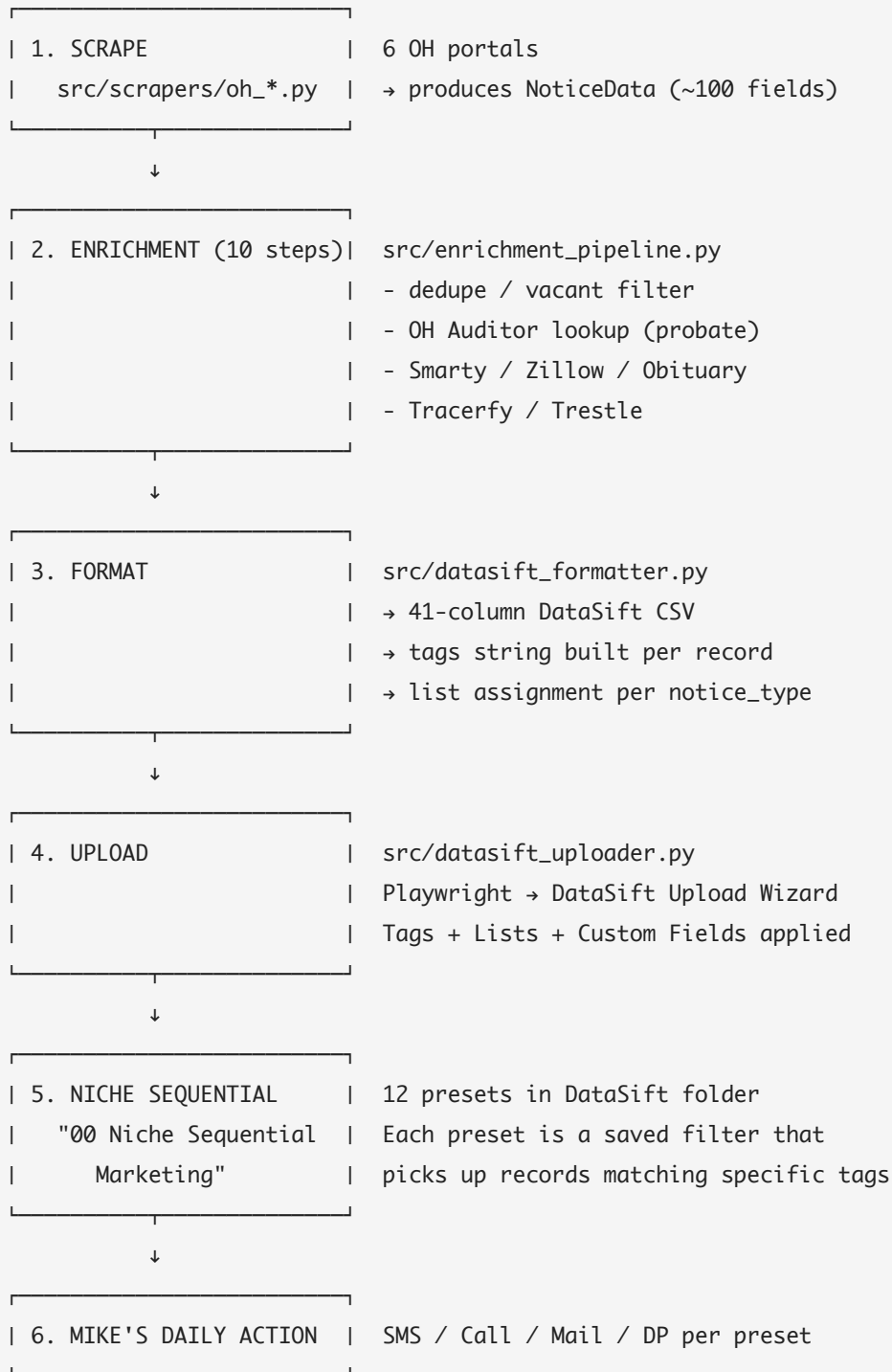


SOP — Tag Flow & Niche Sequential Mapping

Audience: Mike (Data Manager) and Aaron (Owner).

Purpose: Explicit map of how a record moves from courthouse scrape → DataSift → niche sequential preset → Mike's daily action. Use this when a record looks wrong, a tag is missing, or a preset isn't picking up records you expect.

The data flow at 50,000 ft



Each step has a known input → output contract. When something breaks, the symptom (which preset has 0 records, which CSV column is empty) tells you which step failed.

1. NoticeData → Tags map

The `_build_tags()` function in `src/datasift_formatter.py` builds a comma-separated tags string for each record. Every tag below is automatic — there's no manual tagging on upload.

Always applied (every record)

Tag	Source field	When it appears
Courthouse Data	hardcoded	Every record. THIS IS THE NICHE SEQUENTIAL TRIGGER. Without it, no preset picks the record up.

Per-record (driven by NoticeData fields)

Tag	Source	Example values
<code>{notice_type}</code>	<code>notice.notice_type</code>	<code>foreclosure</code> , <code>probate</code> , <code>tax_sale</code> , <code>tax_delinquent</code> , <code>eviction</code> , <code>code_violation</code> , <code>divorce</code>
<code>{county}</code>	<code>notice.county.lower()</code>	<code>franklin</code> , <code>montgomery</code> , <code>greene</code>
<code>{YYYY-MM}</code>	<code>notice.date_added</code> (parsed)	<code>2026-04</code> , <code>2026-05</code>
<code>deceased</code> OR <code>living</code>	<code>notice.owner_deceased</code>	<code>deceased</code> if <code>owner_deceased</code> == <code>"yes"</code> , else <code>living</code>
<code>{confidence}_confidence</code>	<code>notice.dm_confidence</code> (deceased only)	<code>high_confidence</code> , <code>medium_confidence</code> , <code>low_confidence</code>
<code>has_auction</code>	<code>notice.auction_date</code> (future date)	If <code>auction_date</code> is set and in the future
<code>tax_delinquent</code>	<code>notice.tax_delinquent_years</code> (>0)	If parcel is tax delinquent
<code>dm_verified</code>	<code>notice.decision_maker_status ==</code> <code>"verified_living"</code>	DM confirmed living via people search
<code>has_heirs</code> / <code>no_heirs</code>	<code>notice.heir_map_json</code> non-empty	Probate records with completed heir map
<code>has_dm_address</code>	<code>notice.decision_maker_street</code> populated	DM mailing address known
<code>signing_chain_{N}</code>	<code>notice.signing_chain_count</code>	<code>signing_chain_2</code> , <code>signing_chain_3</code> , etc. (number of living signing- authority heirs)
<code>signing_chain_complete</code> / <code>signing_chain_partial</code>	computed	Complete = all heirs phone- verified; partial = some unverified
<code>entity_owned</code>	<code>notice.entity_type</code> set	LLC/Corp/Trust owners
<code>entity_researched</code>	<code>notice.entity_person_name</code> populated	Person identified behind entity
<code>photo_import</code>	<code>notice.raw_text</code> indicates photo source	Records sourced from courthouse phone photos

Tag	Source	Example values
		(dormant — not used in OH today)
<code>skip_traced_{YYYY-MM}</code>	added by DataSift after Skip Trace	Auto-added by DataSift's Skip Trace job after upload

Sample tag string (real Montgomery probate record from yesterday's scrape)

```
Courthouse Data, probate, montgomery, 2026-04, deceased, high_confidence,
has_dm_address, has_heirs, signing_chain_3, signing_chain_complete
```

Tag string for a Franklin foreclosure record (typical)

```
Courthouse Data, foreclosure, franklin, 2026-04, living, has_auction
```

2. NoticeData → DataSift List map

The `NOTICE_TYPE_TO_LIST` mapping in `src/datasift_formatter.py:215` :

<code>notice_type</code> (NoticeData)	DataSift List
<code>foreclosure</code>	Foreclosure
<code>probate</code>	Probate
<code>tax_sale</code>	Tax Sale
<code>tax_delinquent</code>	Tax Delinquent
<code>eviction</code>	Eviction
<code>code_violation</code>	Code Violation
<code>divorce</code>	Divorce

DataSift auto-creates lists from CSV column data. If you ever see records NOT in their expected list, the Lists column wasn't mapped during upload — see [SOP-RED-FLAGS.md](#) Section 4.

Special case for tax_sale records: the formatter additionally pushes them to a "Tax Sale Auction" list when `auction_date` is set, so the same property can appear in two lists for different sequence treatments. See `datasift_formatter.py:691` .

3. NoticeData → DataSift Custom Fields map

Beyond the 11 core auto-mapped fields (Property Street/City/State/ZIP, Owner First/Last Name, Mailing Street/City/State/ZIP, Tags) and Lists/Notes, the formatter populates 13 built-in DataSift fields and 15 custom fields:

Built-in DataSift fields

DataSift field	NoticeData source
Estimated Value	<code>estimated_value</code> (Zestimate)
MSL Status	<code>mls_status</code> (Active / Pending / Sold / Off Market)
Last Sale Date	<code>mls_last_sold_date</code>
Last Sale Price	<code>mls_last_sold_price</code>
Equity Percentage	<code>equity_percent</code>
Tax Delinquent Value	<code>tax_delinquent_amount</code>
Tax Delinquent Year	<code>tax_delinquent_years</code>
Tax Auction Date	<code>auction_date</code> (when tax_sale)
Foreclosure Date	<code>auction_date</code> (when foreclosure)
Probate Open Date	<code>date_added</code> (when probate)
Personal Representative	<code>decision_maker_name</code> (when probate)
Parcel ID	<code>parcel_id</code>
Structure Type / Year Built / SqFt / Beds / Baths / Lot	from Zillow enrichment

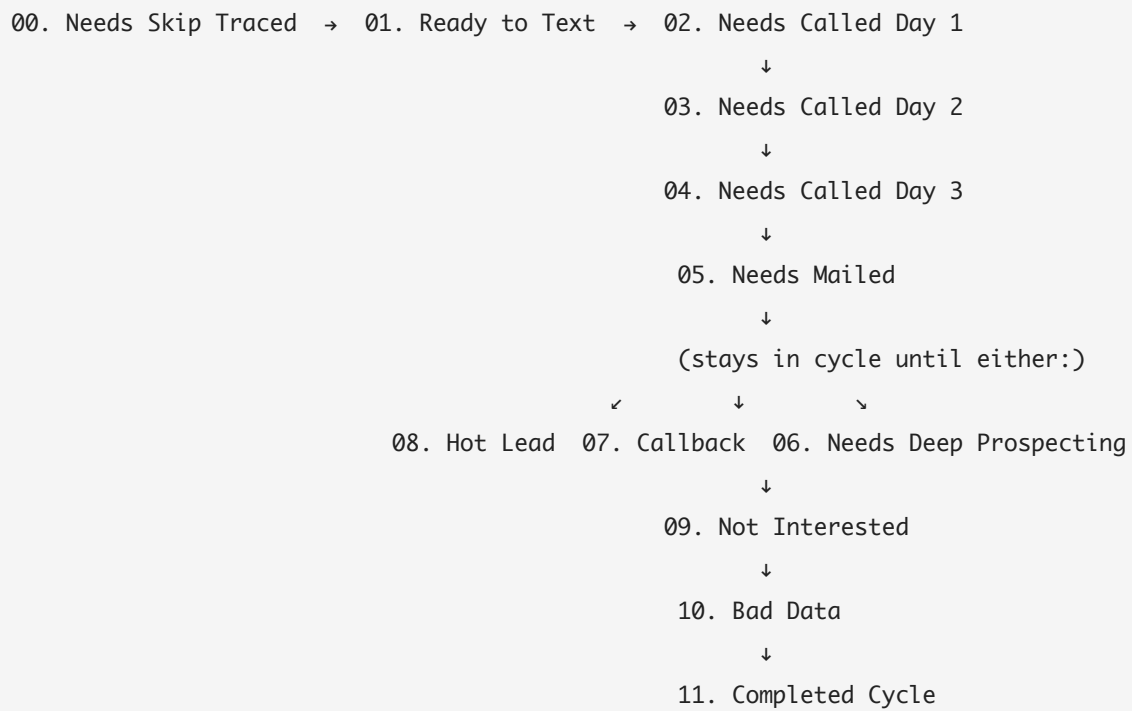
Custom fields (TN Public Notice group — name will get renamed to OH eventually)

DataSift field	NoticeData source
Notice Type	<code>notice_type</code>
County	<code>county</code>
Date Added	<code>date_added</code>
Owner Deceased	<code>owner_deceased</code>
Date of Death	<code>date_of_death</code>
Decedent Name	<code>decedent_name</code>
Decision Maker	<code>decision_maker_name</code>
DM Relationship	<code>decision_maker_relationship</code>
DM Confidence	<code>dm_confidence</code>
DM 2 Name / DM 2 Relationship	<code>decision_maker_2_name</code> / <code>decision_maker_2_relationship</code>
DM 3 Name / DM 3 Relationship	<code>decision_maker_3_name</code> / <code>decision_maker_3_relationship</code>
Obituary URL	<code>obituary_url</code>
Source URL	<code>source_url</code>

4. The 12 Niche Sequential Presets — exact tag matching logic

These live in DataSift under folder **"00 Niche Sequential Marketing"**. Each preset is a saved filter that auto-matches records by tag/field state. Source of truth: `src/niche_sequential.py`.

Cycle order (the Pendulum)



Detailed matching logic

Preset	Tag filter	What Mike does when this fires
00. Needs Skip Traced	<code>Courthouse Data</code> AND no phone	Should never fire if Tracerfy ran cleanly. If it does → enrichment is broken, see SOP-RED-FLAGS.md §3
01. Ready to Text	has phone, phone tier ∈ {Dial First, Dial Second}, NOT <code>sms_sent</code>	Send Day 1 SMS via Launch Control / REISimpli
02. Needs Called Day 1	<code>sms_sent</code> , NOT <code>called_day1</code>	Call all numbers, leave voicemail, log disposition
03. Needs Called Day 2	<code>called_day1</code> , NOT <code>called_day2</code>	Call with alternate script, leave new voicemail
04. Needs Called Day 3	<code>called_day2</code> , NOT <code>called_day3</code>	Final call pass, urgency voicemail, final text
05. Needs Mailed	<code>called_day3</code> , NOT <code>mailed</code>	Export mail-ready CSV, send handwritten letter (~\$1.75/piece)
06. Needs Deep Prospecting	<code>cycle_complete</code> , NOT <code>dp_complete</code> , status ≠ Sold	Route to deep_prospector.py for L1-L3 research
07. Callback Scheduled	<code>callback_scheduled</code>	Call at scheduled time, update disposition
08. Hot Lead	<code>hot</code>	Immediate closer assignment, schedule appointment
09. Not Interested	<code>not_interested</code>	Tag for 90-day follow-up, rotate to different mailer type
10. Bad Data	<code>bad_data</code>	Remove bad phone/address, re-run skip trace
11. Completed Cycle	<code>cycle_complete</code>	(Holding state — pulled into 06. Needs DP)

Tags Mike adds during the day

These tags drive next-day preset matching. They're applied manually in DataSift as Mike works leads:

Tag	When to apply
<code>sms_sent</code>	After SMS goes out (DataSift's SMS tool can auto-tag this on send)
<code>called_day1</code> / <code>called_day2</code> / <code>called_day3</code>	After each call attempt — ALWAYS apply, even on no-answer
<code>mailed</code>	After mail piece sent
<code>callback_scheduled</code>	When the lead requests a callback — set the date in the calendar field
<code>hot</code>	When the lead expresses real interest — closer takes over
<code>not_interested</code>	On a "not interested" disposition
<code>bad_data</code>	When phone is wrong, address is invalid, etc.
<code>cycle_complete</code>	After 05. Needs Mailed → no response → cycle done
<code>dp_complete</code>	After deep prospecting research is finished
<code>Sold</code>	When deal closes — triggers Sold Property Cleanup sequence

5. The 9 Bulk Sequential Presets

Folder: **"01. Bulk Sequential Marketing"**.

Same Pendulum structure as Niche, but for bulk-purchased data (skip-traced lists from external vendors, not your own scraped courthouse data). Records in Niche have the `Courthouse Data` tag — Bulk records do NOT. That's the only difference.

Why both folders exist: Niche records (Courthouse Data) are higher priority — first-to-market, smaller volume, higher conversion. Bulk records are lower priority — backfill when courthouse volume is low. Niche always runs first.

6. Sequence templates (the 26 TCAs)

Folder structure:

DataSift → Sequences

- └─ Lead Management (6 sequences)
- └─ Acquisitions (6 sequences)
- └─ Transactions (6 sequences) ← contains "Sold Property Cleanup"
- └─ Deep Prospecting (4 sequences)
- └─ Default (4 sequences)

These are step-by-step workflow templates DataSift fires when a record matches their trigger. See `src/sequence_templates.py` for the canonical list and `src/datasift_uploader.py` for how they're created/updated programmatically.

The most important one for daily ops:

"Sold Property Cleanup" (Transactions folder)

Trigger: Property Tag added = `Sold`

Condition: Status not already `Sold`

Actions (in order):

1. Change Status → `Sold`
2. Remove from all active lists
3. Clear all assigned tasks
4. Clear assignee

Mike applies the `Sold` tag → this sequence fires automatically. No manual cleanup needed.

7. The full lifecycle of one record (worked example)

Real Montgomery probate record from yesterday's scrape. Walk through each step:

Step 1: Scrapped (07:15 AM, Apify Actor)

```
NoticeData(  
  notice_type="probate",  
  county="Montgomery",  
  state="OH",  
  date_added="2026-04-25",  
  decedent_name="PATRICIA CRIDGE",  
  owner_name="CRIDGE JR, JONATHAN",  
  decision_maker_name="CRIDGE JR, JONATHAN",  
  decision_maker_relationship="executor",  
  decision_maker_street="426 MCNARY AVE",  
  decision_maker_city="DAYTON",  
  decision_maker_state="OH",  
  decision_maker_zip="45417",  
  source_url="https://go.mcoho.org/...casesearchresultx.cfm?TOKEN=ABC123",  
  raw_text="2026EST00786 ESTATE OF PATRICIA CRIDGE...",  
  # address, city, zip - all empty (probate court doesn't have these)  
)
```

Step 2: Enrichment (07:30 AM)

- **Vacant filter:** kept (probate exemption — empty address allowed pre-Auditor lookup)
- **Probate Property Lookup (Step 3c):** Tier 1 search — Montgomery Auditor finds CRIDGE PATRICIA ANN – 2750 WINTON DR (match score 0.667). Address now populated:

```
python notice.address = "2750 WINTON DR" notice.city = "DAYTON" # filled from auditor (or geocoded by Smarty next step) notice.zip = "45419" notice.parcel_id = "K47..."
```
- **Smarty:** standardizes "2750 WINTON DR" → "2750 Winton Dr" + ZIP+4 + lat/long + DPV match code
- **Zillow:** Zestimate \$245,000, equity 78%, 3 bed / 2 bath / 1,540 sqft, year built 1962
- **Obituary:** finds Patricia Cridge obit, DOD 2026-04-15, survived by son Jonathan + daughter Susan + 4 grandchildren. DOD sanity check passes (filed 2026-04-17, DOD 2026-04-15 — within 3 years).
- **Heir verification:** Jonathan (executor) + Susan + grandchildren ranked. Living status verified for top 3 via people search.
- **Tracerfy:** finds 4 phones for Jonathan (3 mobile + 1 landline) + 2 emails. Same for Susan.
- **Trestle phone scoring:** Jonathan's primary mobile = 92 (Dial First). Landline = 67 (Dial Second). Susan's mobile = 88 (Dial First).

Step 3: Tags built (07:50 AM)

```
Courthouse Data, probate, montgomery, 2026-04, deceased, high_confidence,  
has_dm_address, has_heirs, signing_chain_2, signing_chain_complete
```

Step 4: Uploaded to DataSift (08:00 AM)

- List: **Probate**
- All tags applied
- Property Street/City/State/ZIP: 2750 Winton Dr / Dayton / OH / 45419
- Owner First/Last Name: Jonathan / Cridge (the EXECUTOR, not the decedent — per probate contact rule)
- Mailing Street/City/State/ZIP: 426 McNary Ave / Dayton / OH / 45417 (executor's residence)
- Custom fields: Notice Type=probate, Decedent Name=PATRICIA CRIDGE, DM=JONATHAN CRIDGE, DM Relationship=executor, DM Confidence=high, etc.

Step 5: DataSift runs Enrich Property Info + Skip Trace (08:15 AM)

- Enrich pulls SiftMap data (mostly redundant with Zillow, but ensures DataSift has its own native data)
- Skip Trace pulls additional phones (DataSift unlimited plan) — adds 1-2 more numbers, all auto-scored
- New tag added: skip_traced_2026-04 (auto)

Step 6: Niche sequential preset matching (08:25 AM)

- Has phone (multiple) ✓
- Phone tier ∈ {Dial First, Dial Second} ✓
- NOT sms_sent ✓
- → Picked up by **"01. Ready to Text"**

Step 7: Mike's day (9:00 AM)

- Slack post at 8:30 lists this record in "Top 5 by phone score"
- Mike opens DataSift → Filter: "01. Ready to Text" → sees Jonathan
- Reads PDF in output/reports/ (deep prospecting summary)
- Sends Day 1 SMS via Launch Control: "Hi Jonathan, I saw you're handling Patricia Cridge's estate. I noticed there's a property at 2750 Winton Dr in Dayton — wondering if you've thought about what to do with it. Quick reply if interested."
- DataSift auto-tags sms_sent on send
- Tomorrow morning, the same record auto-moves to **"02. Needs Called Day 1"**

8. Validation queries (run these to spot-check)

After the daily run completes, you can run these in DataSift to verify everything flowed:

Validation	Filter in DataSift	Expected count
Today's Courthouse Data records	tag = <code>Courthouse Data</code> AND tag = <code>2026-04-XX</code>	should match Slack summary
Today's deceased records have heir maps	tag = <code>deceased</code> AND tag = <code>has_heirs</code> AND tag = <code>2026-04-XX</code>	should match Slack "deceased owners with heir maps" count
Today's foreclosure records have addresses	list = <code>Foreclosure</code> AND tag = <code>2026-04-XX</code> AND Property Street is NOT empty	should = today's foreclosure count
Today's probate records have DM info	list = <code>Probate</code> AND tag = <code>2026-04-XX</code> AND Decision Maker is NOT empty	should = today's probate count
Today's records ready for SMS	apply preset <code>01. Ready to Text</code>	should match Slack "Tier 1+2" count

9. Reverse lookup — "Why is this record HERE?"

If Mike sees a record in an unexpected preset, work backward:

Record in unexpected preset

↓

Open record → Check Tags field

↓

Compare against the preset's tag filter (Section 4)

↓

Mismatch = wrong tags applied during upload, OR wrong field on NoticeData

If tags wrong:

- Re-upload
- Verify Tags column mapping in wizard

If NoticeData wrong:

- Scraper bug or enrichment bug – check the run log

See also

- [SOP-DAILY-OPERATIONS.md](#) — Mike's morning playbook
- [SOP-RED-FLAGS.md](#) — Diagnostics when things go wrong
- [SOP-REDEMPTION-WINDOW.md](#) — redemption-window niche; new tags `ftm-rw` / `redemption_open` / `redemption_closing` / `redemption_closed` and custom fields Sheriff Sale Held Date, Confirmation Hearing Date, Redemption Window Status, Redemption Days Remaining
- [CLAUDE.md](#) — Full operational reference

- `src/datasift_formatter.py` — canonical tag-building logic (`_build_tags()` at line 226)
- `src/niche_sequential.py` — canonical preset definitions (PRESETS list at line 41)
- `src/sequence_templates.py` — canonical 26 TCA sequence templates
- `src/redemption_watcher.py` — daily code that drives redemption tags + custom fields